



Desenvolvimento de Serviços com Spring Boot

Etapa/Aula 07.1

Professor(a): Flávio Neves

E-mail: flavio.neves@prof.infnet.edu.br

Testagem e Implementação de Testes em Aplicações Spring Boot

Roteiro da Aula

- **Conceitos sobre testes de Softwares**
- **A importância dos testes de Softwares**
- **Impacto nas aplicações/empresas**

Teste de software

- Os testes podem mostrar sempre a presença de erros, mas nunca a ausência (Dijkstra et al., 1972).

Teste de software

- O que é Teste de Software?



Teste de software

- Teste de software é o processo de execução de um produto para determinar se ele atingiu suas especificações e funcionou corretamente no ambiente para o qual foi projetado.



Teste de software

- Existe grande possibilidade de injeção de falhas humanas no processo de desenvolvimento de software;
- O processo de testes faz parte da garantia de qualidade de software;
- Os custos associados às falhas de software justificam um processo de testes cuidadoso e bem planejado.

Teste de software



Defeito=Falha=Anomalia=Bug=Ocorrência

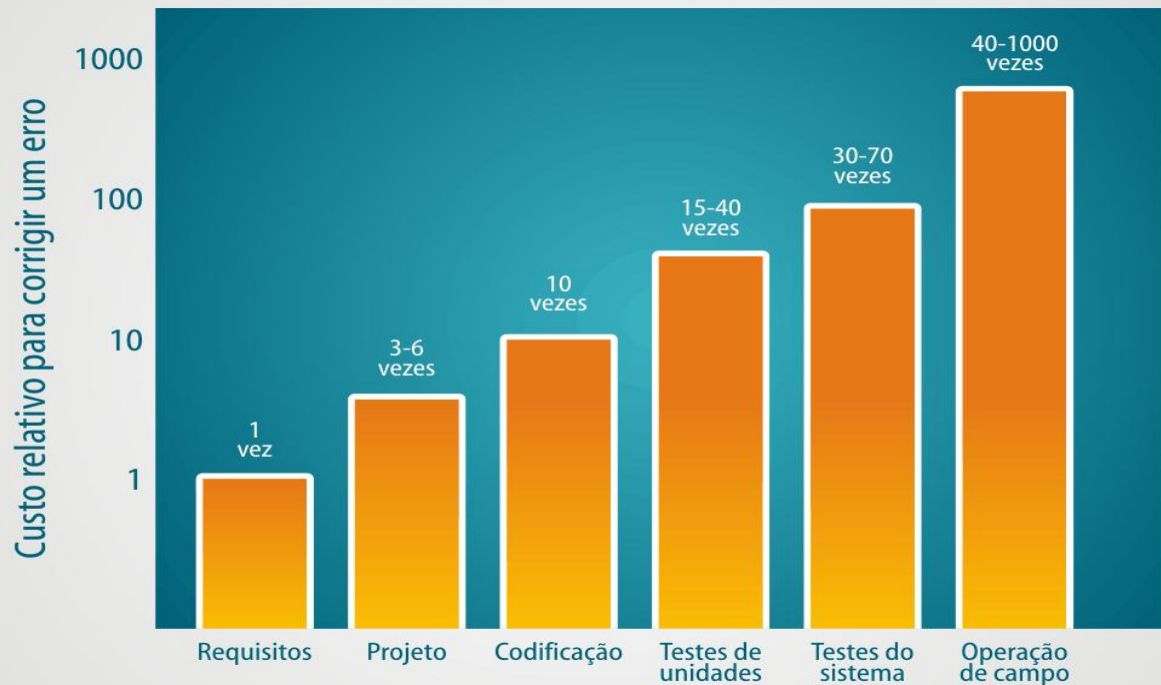
- Existe defeito quando um software ou parte dele:
 - Não funciona, mas os requisitos ou os artefatos indicam que funciona;
 - Faz algo na aplicação em que os requisitos ou os artefatos indicam que não deveria fazer;
 - Faz algo que os requisitos ou os artefatos não indicam;
 - Faz algo que os requisitos ou os artefatos não indicam, porém deveriam indicar;
 - Não funciona adequadamente aos olhos do testador, pois é difícil de entender, de usar ou é lento.

Teste de software

- Pressman afirmou em seu Livro de Engenharia de Software que:
 - O custo do defeito é progressivo, ou seja, encontrar o defeito na fase de engenharia de requisitos custa 1 enquanto encontrar o defeito durante a fase de uso custa 100 vezes mais, então utilizar o teste, reduz custo e não aumenta.

Teste de software

Erros em produção são extremamente caros de corrigir!

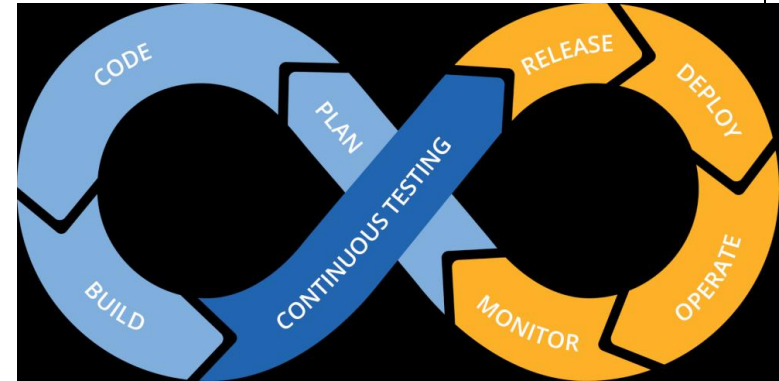


Teste de software - Motivação para os Testes

- Necessidade de produzir software de qualidade!
- Reparar os problemas depois que os produtos forem entregues aos clientes, não é mais aceitável (Ian, 2003)
 - Software que não funcionam adequadamente causam problemas a pessoas e empresas;
 - As falhas causam prejuízos financeiros, risco a vida de seres humanos e à saúde;
 - As falhas causam perda de confiança do cliente.

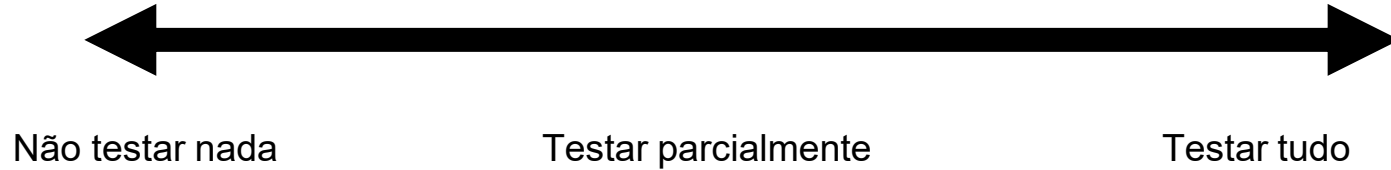
Objetivo do Teste de Software

- Executar um programa para:
 - Encontrar falhas;
 - Medir a qualidade;
 - Garantir a confiança.
- Analisar um programa ou sua documentação para prevenir defeitos;
 - Reduzir custos de manutenção corretiva e retrabalho.

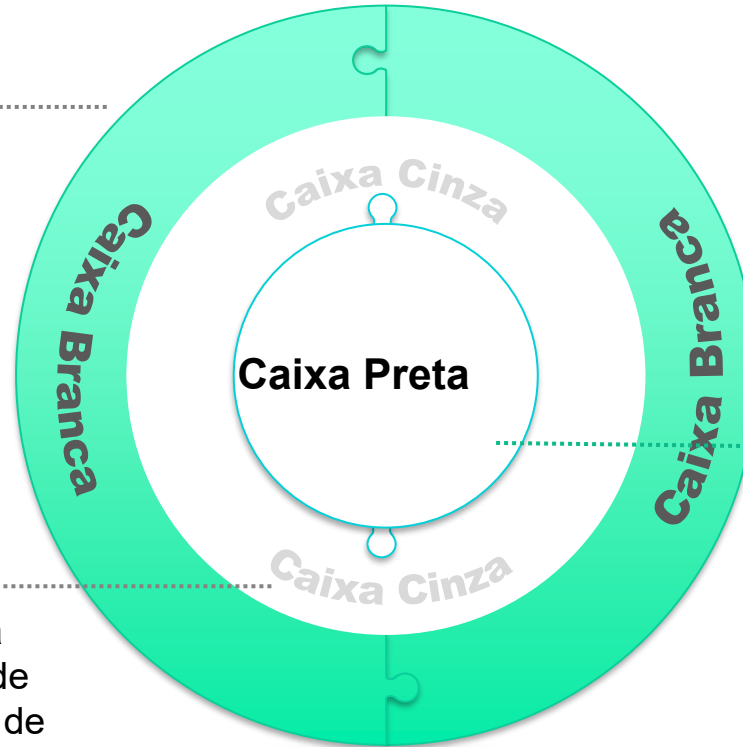


Teste de software

- O que testar?



■ Técnicas popularizadas pelo mercado de software



CAIXA BRANCA - Dentro desta categoria de teste de software o desenvolvedor tem acesso ao código fonte da aplicação e pode construir códigos para efetuar LINKER de componentes.

CAIXA CINZA - Uma definição deste tipo de teste seria um ponto de equilíbrio virtual entre o teste de caixa-branca e o caixa-preta. .

CAIXA PRETA - Neste tipo de teste de software o desenvolvedor dos testes não possui acesso algum ao código fonte do programa.

Categorias de Teste de Software

- Teste de Unidade:
 - Codificação pequenas partes ou unidades do sistema como funções e métodos.
- Teste de Componente:
 - Codificação do componente como um todo e não apenas as suas funções ou métodos.
- Teste de Integração:
 - Integração dos componentes do sistema.
- Teste de Sistema:
 - Atendimento dos requisitos funcionais e não funcionais do sistema testando como se fosse o usuário final.
- Teste de Aceitação:
 - Para permitir ao cliente determinar se aceita ou não o sistema.
- Teste de Regressão:
 - Aplicado na fase de manutenção do sistema, após sua implantação.

Teste de Unidade

Teste de Unidade

- O que testar?
 - Todo comportamento;
 - Condições limites (inferiores e superiores);

Teste de Unidade

- Unidade?
 - Unidade é a menor parte testável de uma aplicação. Em uma linguagem de programação orientada a objetos como o Java, a menor unidade é um método.

Teste de Unidade

- Quando criamos um teste de unidade, simulamos a execução de métodos da classe a ser testada.
- Fazemos isso passando parâmetros (no caso de ser necessário) ao método testado e definimos o resultado que esperamos.
- Se o resultado for igual ao que definimos como esperado, o teste passa. Caso contrário, falha.

Teste de Unidade

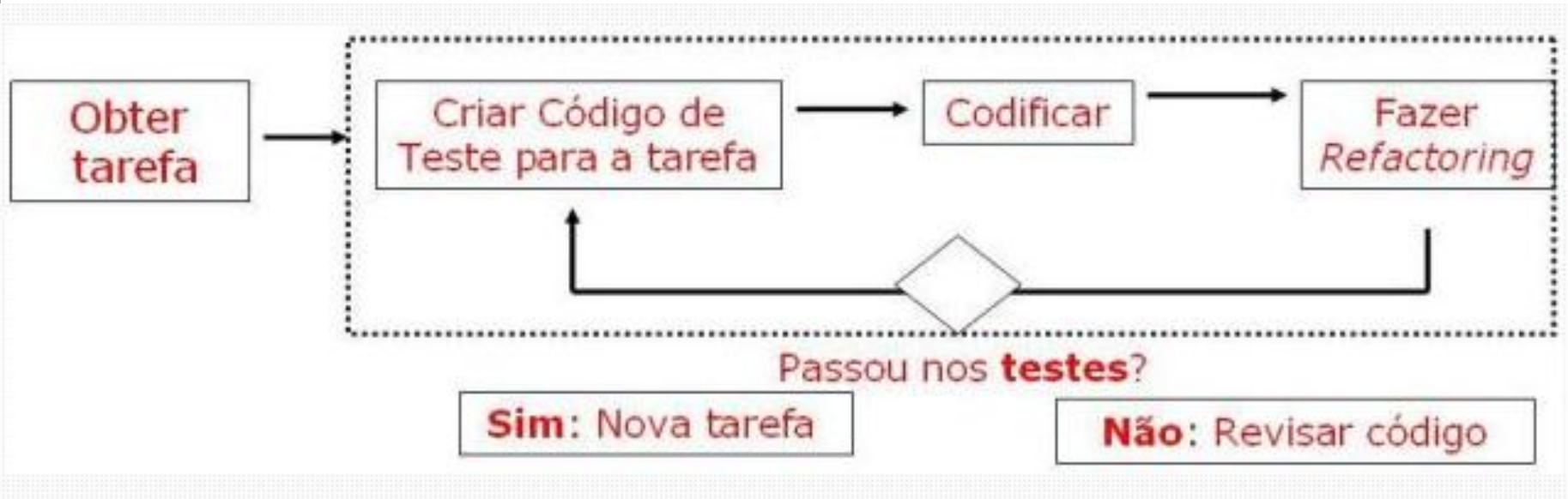
- Vantagens:
 - Previne contra o aparecimento de “bugs” provenientes de códigos mal escritos;
 - Código testado é mais confiável;
 - Permite alterações sem medo;
 - Testa situações de sucesso e de falha;
 - Resulta em outras práticas XP como: Código coletivo, refatoração, integração contínua;
 - Serve como métrica do projeto (testes == requisitos).
 - Gera e preserva um “conhecimento” sobre as regras de negócios do projeto.

Teste de Unidade

- Este tipo de teste é de responsabilidade do próprio desenvolvedor durante a implementação do sistema;
- Pode ser escrito antes ou depois de implementado o objeto alvo do teste.

Teste de Unidade

- Processo de testes unitários dentro de seu projeto



Organização das classes de testes

Aplicação

SuaClasse1

-atributo1 : string
-atributo2 : string
+Método1()

SuaClasse2

-atributo1 : string
-atributo2 : string
+Método1()

SuaClasse3

-atributo1 : string
-atributo2 : string
+Método1()

<<Classe Java>>

- Propriedade Coletiva
- Padrões de Projeto
- Design Simples

Testes

ClasseTeste1

+testValor1()

ClasseTeste2

+testValor1()
+testValor2()

ClasseTeste3

+testValor2()

<<Teste Case>>

- Teste Unitário
- Critério de Aceitação
- Refatoração

TesteSuite(All Teste)

+suite()

<<Teste Suite>>

- Integração Contínua
- Pequenas Versões

É possível aplicar os tradicionais testes de regressão

Teste de Unidade: O que testar?

- A principal regra para saber o que testar é: “Tenha criatividade para imaginar as possibilidades de testes”;
- Comece pelas mais simples e deixe os testes “complexos” para o final;
- Use apenas dados suficientes (não teste 10 condições se três forem suficientes);
- Não teste métodos triviais, tipo get e set;
- No caso de um método set, só faça o teste caso haja validação de dados;
- Achou um bug? Não conserte sem antes escrever um teste que o pegue (se você não o fizer, ele volta).

MovimentoFinanceiro

-ID : int

-TipoES : char

-DataEmissao : Date

-Cliente : int

-DataVencimento : Date

-DataPagamento : Date

-ValorOriginal : float

-ValorJuros : float

-ValorPagamento : float

+GeraID(ValorIncremento : int) : int

+GeraVencimento(dtEmissao : Date) : Date

+CalculaJuros(ValorOriginal : float) : float

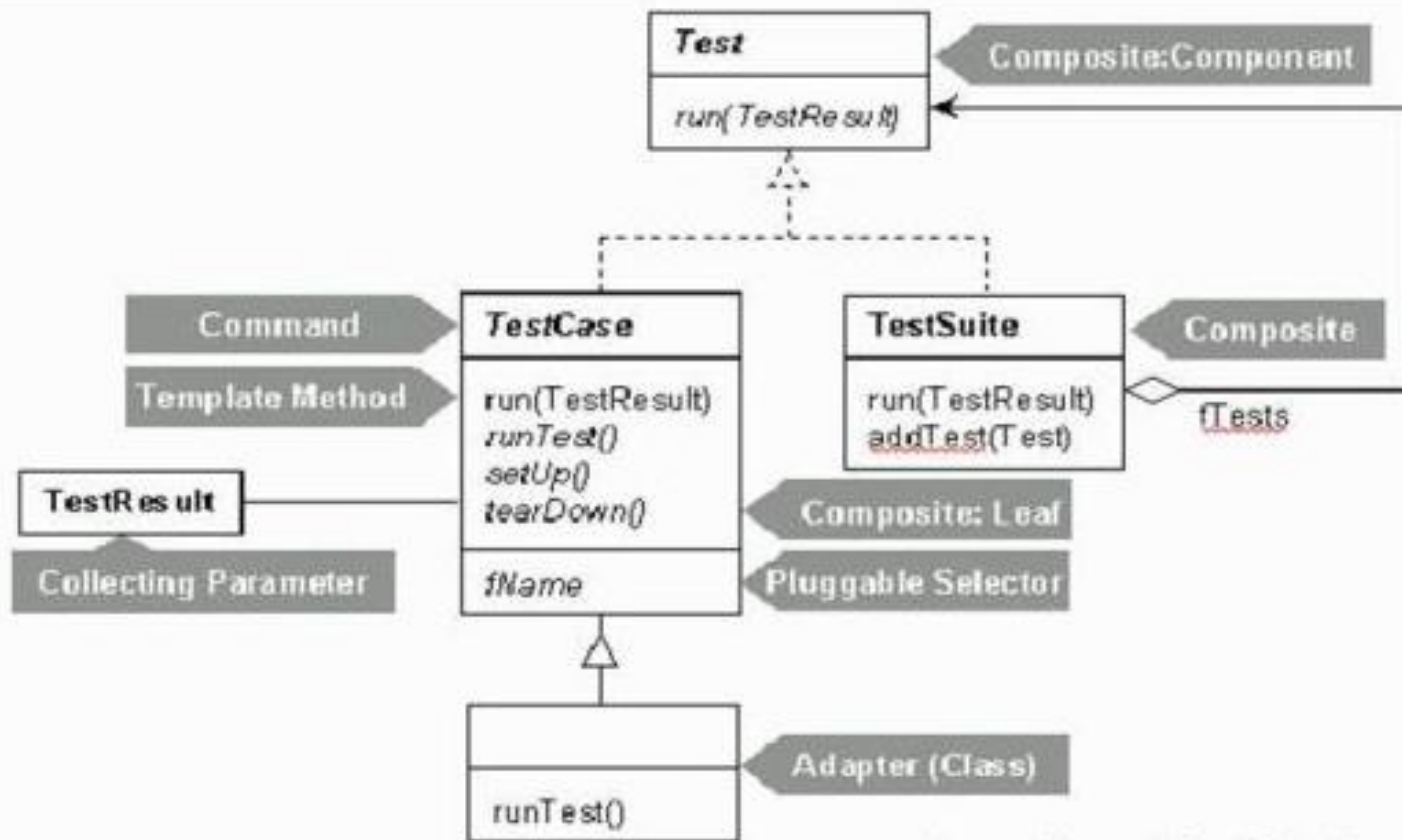
+CalculaValorPagamento(ValorOriginal : float, ValorJuros : float) : float

Teste de Unidade - JUnit

- O JUnit (junit.org) é um framework open-source muito simples para facilitar a criação destes testes de unidade e em especial sua execução.
- Ele possui alguns métodos que tornam seu código de teste bem legível e fácil de fazer as asserções.

Teste de Unidade - JUnit

- Por que usar JUnit?
 - Casos de teste são constituídos por um ou mais métodos, sendo que estes podem estar agrupados em suítes de teste;
 - Ele Fornece uma API (conjunto de classes) para construir os testes e Aplicações gráficas e em modo console para executar os testes criados;
 - Pode verificar se cada unidade de código funciona da forma esperada.
 - Facilita a criação, execução automática de testes e a apresentação dos resultados;
 - É Orientado a Objeto.



Fonte: Manual do JUnit (Cooks Tour)

Executando testes com Junit

```
ContaCorrente.java
1 package com.flavio.aplicacao;
2
3 public class ContaCorrente {
4
5     private double saldo;
6
7     public void depositar(double valorDeposito) {
8         saldo += valorDeposito;
9
10        System.out.println("Deposito realizado com sucesso. ");
11    }
12
13    public void sacar(double valorSaque) {
14        if (valorSaque <= this.saldo) {
15            saldo -= valorSaque;
16            System.out.println("Saque realizado com sucesso. ");
17        } else {
18            System.out.println("Saldo insuficiente, saque não efetivado. ");
19        }
20    }
21
22    public double consultaSaldo() {
23        return saldo;
24    }
25
26 }
27
28 }
```

Executando testes com JUnit

- Escrevendo o primeiro teste unitário com JUnit

The screenshot shows an IDE with the following components:

- Left Panel (Project Explorer):** Shows the project structure with folders 'testes' and 'src'. Under 'src', there is a package 'com.flavio.test' containing 'ContaCorrenteTest.java'.
- Top Panel (Tabs):** Shows 'ContaCorrente.java', 'RealizarDepositoMainTest.java', and 'ContaCorrenteTest.java'.
- Code Editor:** Displays the code for 'ContaCorrenteTest.java':

```
1 package com.flavio.test;
2
3 import static org.junit.jupiter.api.Assertions.*;
4 import org.junit.jupiter.api.Test;
5 import com.flavio.aplicacao.ContaCorrente;
6
7
8 class ContaCorrenteTest {
9
10     @Test
11     public void realizarDepositoEmContaCorrente() {
12         double valorDeposito = 200.0;
13         double saldoEsperado = 200.0;
14         ContaCorrente cc = new ContaCorrente();
15         cc.depositar(valorDeposito);
16         double saldoDaConta = cc.consultaSaldo();
17
18         assertEquals(saldoEsperado, saldoDaConta, 0.0001);
19     }
20 }
21
22 }
```
- Annotations:**
 - A red arrow points from the `@Test` annotation to a box: "Anotação JUnit para definir um método de teste".
 - A bracket groups the test method body, with an arrow pointing to a box: "Define um cenário de teste".
 - An arrow points from the `assertEquals` call to a box: "Método estático da classe Assert para validar o teste".
- Bottom Panel (Test Results):** Shows "Finished after 0,394 seconds" and a summary: "Runs: 1/1", "Errors: 0", "Failures: 0". A green progress bar is shown below.

Criação de testes unitários

- De maneira generalizada o desenvolvedor
 1. Pensar primeiro em um cenário (um valor a ser testados)
 2. Executa a ação (executa os método da classe)
 3. Valida a saída (define a asserção)



- Test-Driven Development by Example (TDD Desenvolvimento Guiado por Testes)

Referências

- Pressman, R., & Maxim, B. (2016). *Engenharia de Software-8ª Edição*. McGraw Hill Brasil.
- Delamaro, M., Jino, M., & Maldonado, J. (2013). *Introdução ao teste de software*. Elsevier Brasil.